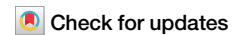


<https://doi.org/10.1038/s44335-026-00063-7>

Improving deep neural network performance through sampling



Lakshmi A. Ghantasala^{1,2}✉, Ming-Che Li¹, Risi Jaiswal¹, Behtash Behin-Aein², Joseph Makin¹, Shreyas Sen¹ & Supriyo Datta¹

Energy-efficient sampling with probabilistic neurons or *p-bits* has been demonstrated in the context of Boltzmann machines, and it is natural to ask if these approaches can be extended to the field of generative AI, where energy costs have become prohibitively large. However, this very active field is dominated by feedforward deep neural networks (DNNs), which primarily use multi-bit deterministic neurons with no role for sampling. In this paper, we first show that it is feasible to obtain superior accuracy through the use of multiple samples generated by probabilistic networks. This possibility raises the question of which option is energetically preferable for improving accuracy: generating more samples or adding more bits to a single deterministic sample. We provide a simple expression that can be used to estimate these energy tradeoffs and illustrate it with results for different algorithms and architectures.

The use of probabilistic neurons or *pbits* comes naturally in the context of Boltzmann machines, where the loss function is expressed in terms of binary stochastic variables, aka classical spins. This has given rise to the vast field of Ising computing, where significant energy advantages have been demonstrated through ASICs designed to solve optimization and sampling problems (see, for example, Li et al. (2025)¹, ISSCC, and references therein). By contrast, the field of generative AI is dominated by feedforward deep neural networks (DNNs), which primarily use multi-bit deterministic neurons with no role for sampling.

Our preliminary results suggest that it may be possible to obtain comparable or even superior accuracy using multiple samples obtained from probabilistic neurons. This possibility immediately raises the question of the energy cost of generating T samples, and the answer depends on various algorithmic and architectural considerations, such as the number of samples in question, the bit precision, and connective fan-out per neuron, among others.

While experimental validation in this work is carried out on FPGA hardware for accessibility and measurement fidelity, the p-DNN architecture is explicitly designed around the p-bit abstraction originally introduced for unconventional computing substrates such as stochastic MTJs, memristive devices, and in-memory compute fabrics. The elementary operation defined in this work mirrors the p-bit synapse-neuron primitive used in prior Ising and Boltzmann machine accelerators, enabling direct mapping onto non-von-Neumann and physics-driven hardware. In this sense, the FPGA serves as a functional emulator of unconventional substrates rather than the target platform.

Our primary objective in this paper is to provide a simple framework that can be used to estimate these energy tradeoffs for different implementations and under different circumstances. We define a universal elementary operation, sketched in Fig. 1a, whose energy cost ϵ_{EO} can be written as (see Fig. 1b)

$$\epsilon_{EO} = \underbrace{nb_w \epsilon_{wM}}_{\text{read weights}} + \underbrace{(n+1)b_a \epsilon_{aM}}_{\text{read act.s and write out}} + \underbrace{\epsilon_S(n, b_a, b_w)}_{\text{synapse energy}} + \underbrace{\epsilon_N}_{\text{neuron energy}} \quad (1)$$

where ϵ_{wM} and ϵ_{aM} represent the energy to access one bit from the weight memory and the activation memory, respectively, ϵ_S is the energy needed to compute the function W over n weights, b_w and b_a are the bits used to represent weights and activations, respectively, and ϵ_N is the energy needed to generate a neuron output based on the output of W . The energy to write the result to activation memory is assumed equal to ϵ_{aM} .

Offhand, one might assume that collecting T samples would cost T times as much energy, but this is not true if the samples are all generated from a single reading of the weights without reloading. In this case, Eq. (1) becomes

$$\epsilon_{EO} = nb_w \epsilon_{wM} + T[(n+1)b_a \epsilon_{aM} + \epsilon_S(n, b_a, b_w) + \epsilon_N] \quad (2)$$

showing that the energy cost of T samples can be minimal if ϵ_{EO} is dominated by the energy needed to load weights (first term).

We emphasize that we view this as a first step towards evaluating the possible role of probabilistic sampling in the machine learning space. Much

¹Elmore School of Electrical and Computer Engineering, Purdue University, West Lafayette, IN, USA. ²Ludwig Computing, Mill Valley, CA, USA.

✉ e-mail: alasghantasala@gmail.com

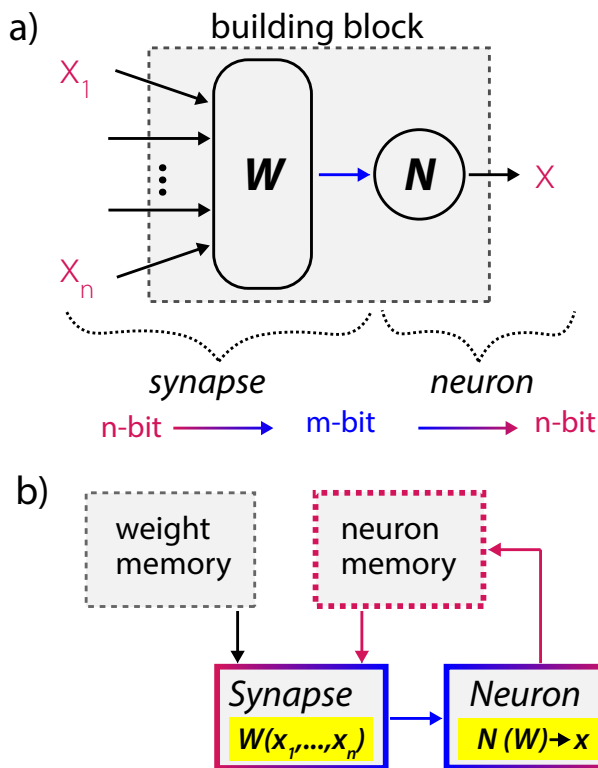


Fig. 1 | The fundamental probabilistic building block and its core operations. **a** Building block of p-circuits, **b** System architecture used to implement the building block, based on which the energy ϵ per elementary operation (Eq. (1)) is written.

depends on how many samples (T) are needed to achieve a significant enhancement in accuracy. The answer will clearly depend on the algorithm used to generate the samples, where the possibilities are fairly open-ended at this time. Each algorithm will then need to be evaluated with an end-to-end evaluation framework to estimate its energy cost. Only then can we answer the question of whether, for a given performance quality and energy cost, it is preferable to use a deterministic n -bit DNN or an m -bit DNN ($m < n$) with T samples. All we provide in this paper is (1) proof-of-concept that small values of T can lead to significant improvements in performance, and (2) a framework for evaluating energy costs (Eq. (2)) with illustrative examples.

Results

P-bits for DNNs: P-DNNs

Let us first discuss *two possible ways* to incorporate p-bits into deep neural networks (DNNs) to construct probabilistic or p-DNNs.

Sample-aware training of DNNs. A standard approach to realize low-bit DNNs is through *quantization* — *aware* training, which generally is applied to weights, though the idea can extend to activations as well². We follow a similar approach, but instead train with multiple samples, following a *sample-aware training* scheme as shown in Fig. 2c. This approach involves using the loss value computed from the average of s samples to perform the training by backpropagation³. The idea then is to *replace* the activations of a traditional DNN with stochastic activations and then train the model to take into account the stochasticity.

Through sample-aware training, the multiply-accumulate operation, traditionally a computation bottleneck in DNNs⁴, is simplified to being between a multi-bit digital weight and a single-bit activation. This dramatically reduces the compute energy, where compute is the sum of synapse and neuron energies, though one has to be wary that drawing multiple samples will again increase that compute energy. We will show that 2 samples is enough to improve accuracy.

We apply this sample-aware training scheme to two p-DNNs, a custom VGG5 model (see Fig. 9b) for image classification trained on the CIFAR10 dataset, and a custom VAE model (see Fig. 9a) for image generation trained on the CelebA dataset.

Image classification: We show that one sample of a 1-bit activation p-DNN is enough to match deterministic accuracy, with 2 samples outperforming. Since this amounts to adding a minor compute energy (and *not* memory energy), the overall energy cost is quite small for the accuracy bump, as seen in Fig. 6b. Additional samples from the p-DNN model lead to higher accuracy, roughly matching 3-bit deterministic accuracy at 10 samples as seen in Fig. 4a. The colored lines in Fig. 4a use multi-sample training and inference by averaging *at the end* of the model. The dotted black line shows averaging test-time samples *at each layer* in the model, with the same weights as those in the deterministic baseline. Interestingly, while the algorithm improves accuracy even at 1-sample inference of a model trained at 45 samples for the classification task, it only improves inference at the same number of samples for the generation task, which we will discuss next.

Image generation: As the baseline, we use a 12-layer Variational Autoencoder DNN using 32-bit weights and sigmoid activations trained to generate celebrity images from a random input (Fig. 3a). Replacing the 32-bit activations with p-bits we get very noisy images, but averaging over 100 samples creates a hint of recognizable facial images (Fig. 3b). Retraining the weights using the actual nonlinear element instead of the continuous version, the quality of images improves significantly (Fig. 3c). Replacing the p-bit activations in the final layer with continuous sigmoid activations marks an improvement, even at 1 sample (Fig. 3d, top row). High-quality images comparable to the original baseline are obtained (see final row of Fig. 3d) with sample-aware training. Figure 3 compares the image quality for different schemes quantitatively using the Fréchet-Inception Distance (FID). A key point to note is that better performance is obtained when the number of samples used in testing *matches* that used in re-training. Furthermore, with sample-aware training, images generated with p-bits can reach very close to 32b FID scores.

Surprisingly, even simple averages at each p-bit output using weights from a 32b deterministic model seem to produce recognizable face images at just 100 samples, as opposed to the expected $O(2^{64})$. It would seem that though standard error applies for *each operation*, it doesn't necessarily apply to the *end application* (e.g. image generation), which has some robustness built into it through the training process. This extends to classification problems as well, as we'll discuss next.

Implementation details for both experiments are included in the appendix.

Sampling noisy DNNs. While sample-aware training requires re-training a model with stochastic activations, there may be ways to take advantage of sampling on DNNs even without retraining, through added noise. By adding some noise to a deterministically trained model, it is possible to generate “noisy samples” that, when averaged, produce lower loss results than a standard model.

We apply this approach to the activations of a 32b model trained for CIFAR-10 classification. The model is run at 4b in a standard post-training quantization fashion⁵, with group size 64, as well as quantized after adding some noise to the original model. With just two samples, the sampled model produces better results, shown in Fig. 4c. The improvement shown here is modest, but what is surprising is that the improvement is monotonic. One might have expected that adding noise to a deterministically trained model would randomly improve or degrade performance with samples, but that does not seem to be the case.

We've now discussed two potential approaches to taking advantage of sampling via p-DNNs: a model trained with p-bit activations, and a model with noise added to standard activations. In problems of this type, our aim is not to recreate the activations A precisely but rather to minimize the loss function $\mathcal{L}$ characterizing the mapping of probability distributions. This is facilitated by the fact that, at the optimum, the partial derivatives $\frac{\partial \mathcal{L}}{\partial A}$ are on average zero, so the loss is insensitive to small perturbations in the

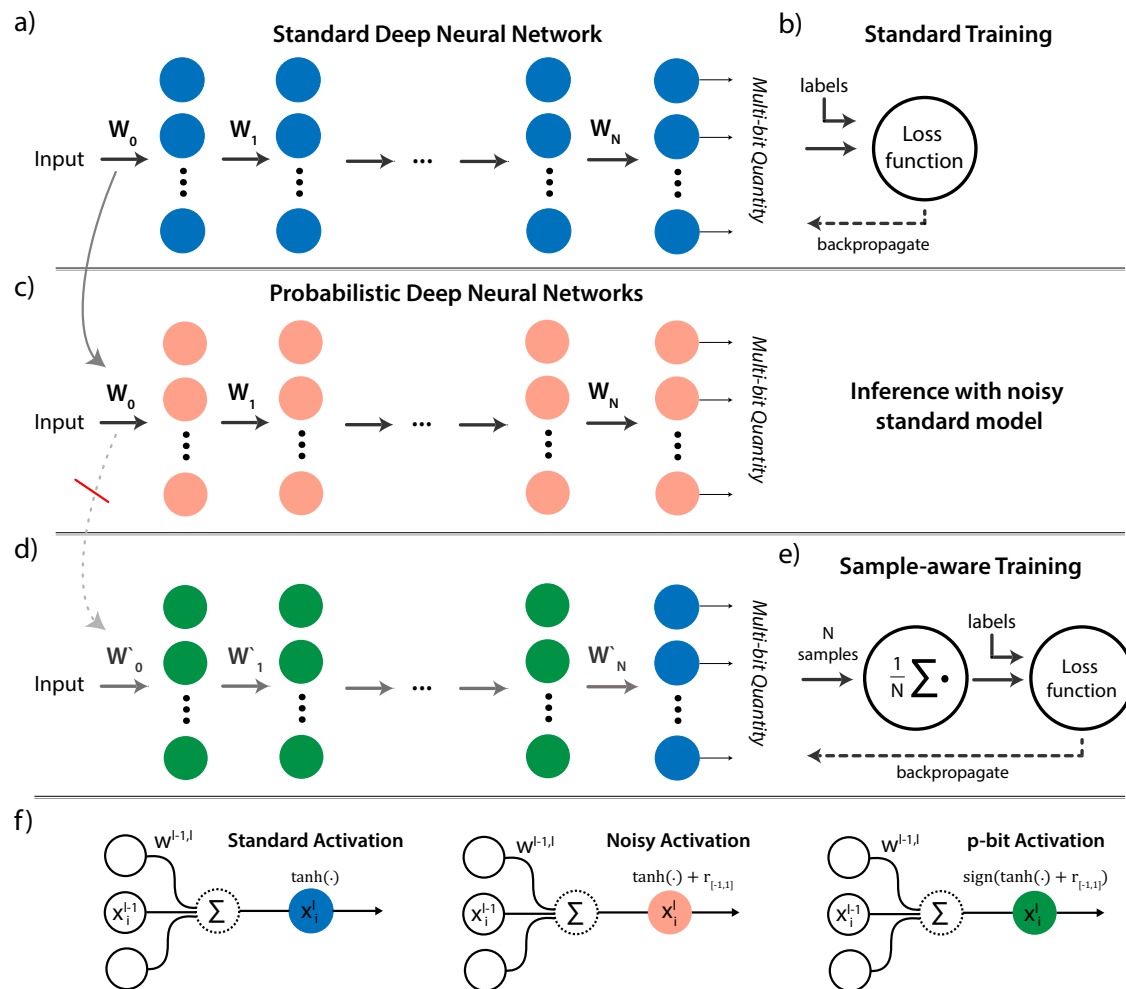


Fig. 2 | Comparison of architectures and training schemes for standard deterministic and probabilistic deep neural networks (p-DNNs). **a** Standard DNNs are implemented with activation functions like tanh or ReLU, and **b** are generally trained following backpropagation. PDNNs can come in different forms, broadly

referring to DNNs with stochasticity infused into each forward pass. **c** Shows a model that integrates stochasticity via noisy activations, while **d** shows a model that replaces the activations with pbits. **e** Shows the sample-aware training scheme that dramatically improves performance for pDNNs. **f** Specifies the activation functions.

activations. We can get better results by taking into account more samples to recover any loss of $\mathcal{L}$ due to the noise we add to A . In the next section, we will discuss the energy cost of each implementation and how they (and potentially any other method) fit into the energy framework outlined by Eq. (1).

Contrast with stochastic bitstreaming. It should be noted that using p-bits in place of continuous neurons as described above is very different from the well-known use of stochastic bitstreams to represent continuous signals. With stochastic bitstreams, the aim is to generate streams of single bits whose mean value matches the original 32-bit value. Since the mean of a non-linear function is not equal to the function of the mean, sample averaging should be performed individually for *each linear operation at each layer*. Incidentally, we note that the stochastic bitstreaming approach does achieve results that exceed statistical expectations derived from its standard error scaling, though not enough to match the presented mechanisms.

Energy analysis of sampling systems

The solution could be reached faster at the same power level if the energy per elementary operation ϵ_{EO} could be lowered, for example, through the use of in-memory computing and/or analog addition followed by a sampler that uses the analog sum to generate a binary output. These are part of the hardware design, while the quantities N , n and T are set by algorithmic

considerations. Let us elaborate a little on the different energy components of the elementary operation described by Eq. (1):

The core components. *Memory access energy, ϵ_{wM} , ϵ_{aM} :* Most large-scale applications in machine learning are memory bottlenecked in latency and energy, making this component especially valuable in an end-to-end analysis of probabilistic solutions. In ref. 1, both weights and p-bit values were stored in local registers, making their read energies identical and relatively low in the tens of fJ range. With large problems, it may not be possible to store all weights in registers, requiring SRAM or even DRAM with much larger read energies. Separating weight and activation memory is useful in that p-bits as activations are binary, and may be stored in closer-to-compute memory than weights, which may be orders of magnitude larger overall. In that case, in-memory computing options can be employed to minimize read energies, which can otherwise dominate the other components (Fig. 6). In Eq. (1), memory access energy involves reading n weights, reading n activations, and writing the output activation of the building block back to memory.

Synapse energy, $\epsilon_S(n, b_w, b_a)$: The function W is usually linear: $w_1x_1 + w_2x_2 + \dots + w_nx_n$ for b_w -bit w_i quantities. Usually, a multiply-accumulate unit (MAC) is used to evaluate W , but for p-bit activations, since $x_1, \dots, x_n$ become binary variables, the function W only requires an accumulate (AC) function that selectively sums a subset of the weights $w_1, \dots, w_n$. Based on n , there may be

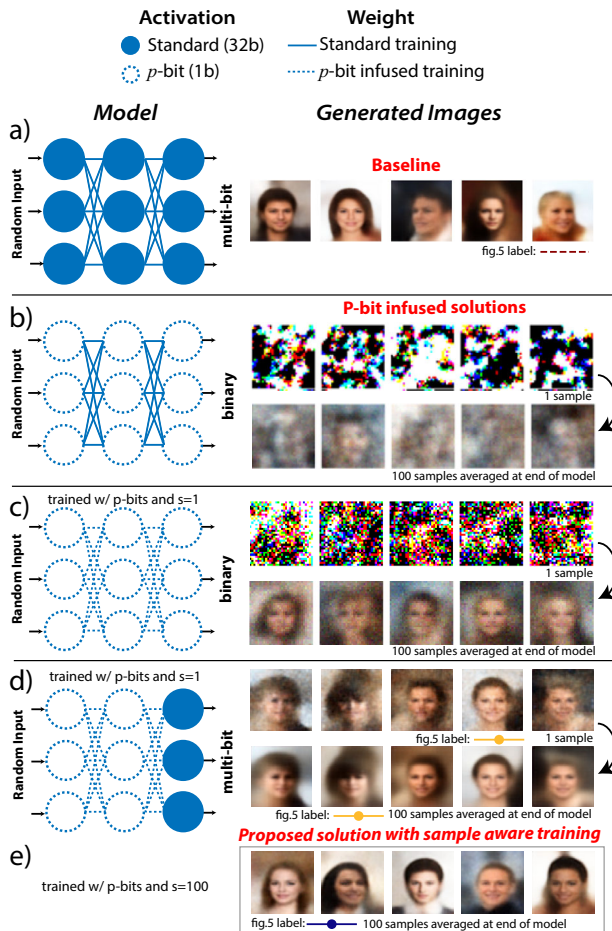


Fig. 3 | Images generated via a traditional variational autoencoder compared with various probabilistic alternatives of the same architecture. a standard DNN inference. **b** Model with pbits replacing all activations. **c** Model that is retrained with pbit activations. **d** Model that is retrained with pbit activations and an analog last layer. **e** Proposed sample aware training scheme employed on (d).

architectural considerations to take into account. For instance, the problems in ref. 1 use $n = 6$, which is relatively small so that the function W can be read off a look-up table (LUT) with $2^n = 64$ entries. For p-bit building blocks where n is too large for the computation to be implemented with a LUT, $\epsilon_S \sim n\epsilon_{add}(b_w)$, where $\epsilon_{add}(b_w)$ is the energy for adding two b_w bit numbers via an adder. For deterministic building blocks, $\epsilon_S \sim n\epsilon_{mul}(b_a, b_w) + \epsilon_{add}(b_a)$, where ϵ_{mul} is the energy for multiplying a b_a bit neuron with a b_w bit weight. These products then need to be accumulated with the add.

Neuron energy, ϵ_N : The neuron energy is defined as the energy needed to apply some non-linear function to an input. For p-bit based systems, the neuron may be implemented following the function $b = \text{sign}(\tanh(W) - \text{rand}_{[-1, +1]})$ where $\text{rand}_{[-1, +1]}$ is a random number distributed uniformly between -1 and $+1$. The p-bit based neuron energy then has three sub-components: (1) compute the activation (ϵ_{act}), (2) generate $\text{rand}_{[-1, +1]}$ (ϵ_{RNG}), and (3) compare the two to generate a binary output ($\epsilon_{compare}$). Note that, unlike the memory read and compute energies, the sampling energy for p-bit based systems is independent of n and becomes relatively unimportant for large values of n , i.e., the neuron fan-in. In deterministic systems, there would be no RNG; for instance, in machine learning models, the neuron generally takes the form of some activation like ReLU or swiglu. For more complicated models, the neuron may take the form of softmax or normalization, which we will discuss later.

We define these components not to say that all probabilistic solutions should map to a single rigid formula, but rather to establish a framework with which to build custom energy functions for unique situations.

Figure 5 reveals that for low-K Boltzmann machine architectures, compute energy dominates the overall system. This motivates replacing the compute unit in these systems, which naturally requires sampling, with real p-bit hardware like s-MTJs or zener diodes to achieve meaningful end-to-end energy efficiency improvements. We will show in the coming sections that DNNs are not compute-dominated, but rather memory-dominated. We will argue that in such systems, minor increases in compute are almost negligible in the end-to-end pipeline in terms of energy, but can have a profound impact on the accuracy metrics of the task.

65 nm ASIC for Quantum Monte-carlo. Let us now discuss how the energy equation applies to a standard Boltzmann machine implemented with 65 nm CMOS technology¹, which was used to solve a diverse set of problems having very different values of N , problem size, and T , sample requirement, but in each case the energy per elementary operation was about the same: $\epsilon_p \sim 0.5pf$. The energy split between the components of Eq. (1) is depicted in Fig. 5 for a specific configuration of hyperparameters pertaining to the QMC problem solved in ref. 1.

A fourth energy component acting as a design-specific catch-all term is added to Fig. 5 to include any control logic, I/O, clock distribution, routing, or other overheads. For the ASIC in ref. 1, this design-specific overhead amounted to 25% of the overall energy consumption, but we have not included this component in Eq. (1), since it is often not fundamental to the underlying algorithm or architecture.

Figure 5 shows that the Quantum-Monte Carlo problem is dominated by computational energy, where we can define compute as the sum of synapse and neuron energies. In such systems, it is clear that introducing additional compute would require careful consideration. In the next example, we will show that additional compute is much less costly for DNNs, and can be taken advantage of for notable accuracy gains.

DNN's with sampling, p-DNN's. The energy E for an N -neuron DNN is N times the energy per elementary operation (ϵ_{EO}) in Eq. (2):

$$E = N(nb_w \epsilon_{wM} + T(n+1)b_a \epsilon_{aM} + T\epsilon_S(n, b_a, b_w) + T\epsilon_N) \quad (3)$$

where a p-bit implementation could average over T samples to improve prediction quality. We bring T inside the elementary operation, as multiple samples can be generated without reloading the weights. This is why drawing multiple samples is significantly cheaper relative to drawing a single sample, where the weight loading energy cannot be amortized.

We can consider two cases for Eq. (3), a p-bit case and a standard case, where the p-bit case mimics the sample-aware training models (i.e., activations are replaced with p-bits). For the p-bit case, $\epsilon_N \rightarrow \epsilon_{act} + \epsilon_{RNG} + \epsilon_{compare}$ and $\epsilon_S \rightarrow n\epsilon_{add}$, while for the deterministic case, $\epsilon_N \rightarrow \epsilon_{act}$ and $\epsilon_S \rightarrow n\epsilon_{mul} + n\epsilon_{add}$. The energy gain can then be defined as $gain = \frac{E_d}{E_p}$ where E_d is the energy draw for the deterministic DNN and E_p for the p-DNN. The n used is the average fan-in weighted by the number of parameters per layer in the VGG5 model in Fig. 9b. The standard DNN building block multiplies n weights (each b_w bits) with n activations (each b_a bits), while the p-bit implementation only sums up a selected subset of n weights (each b_w bits), corresponding to activations with $b_a=1$.

By plugging in the values for these quantities, recorded in Table 1 and visualized in Fig. 7, we can estimate the overall energy draw of a custom hardware built to run inference using these models, which is shown in Fig. 6. These quantities are simulated using the same platform and PDK used to design the ASIC in ref. 1. Note that this analysis compares p-DNNs with DNNs at the same weight bit width.

The following are some critical insights we can gather from this experiment.

- In contrast to the QMC chip results in Fig. 5, the memory energy dominates the overall ML pipeline, even for small sized models (~2M parameters), as seen in Fig. 6a. For applications requiring DRAM, which are most production scale LLMs today (100B+ parameter models), adding some compute for additional performance would be

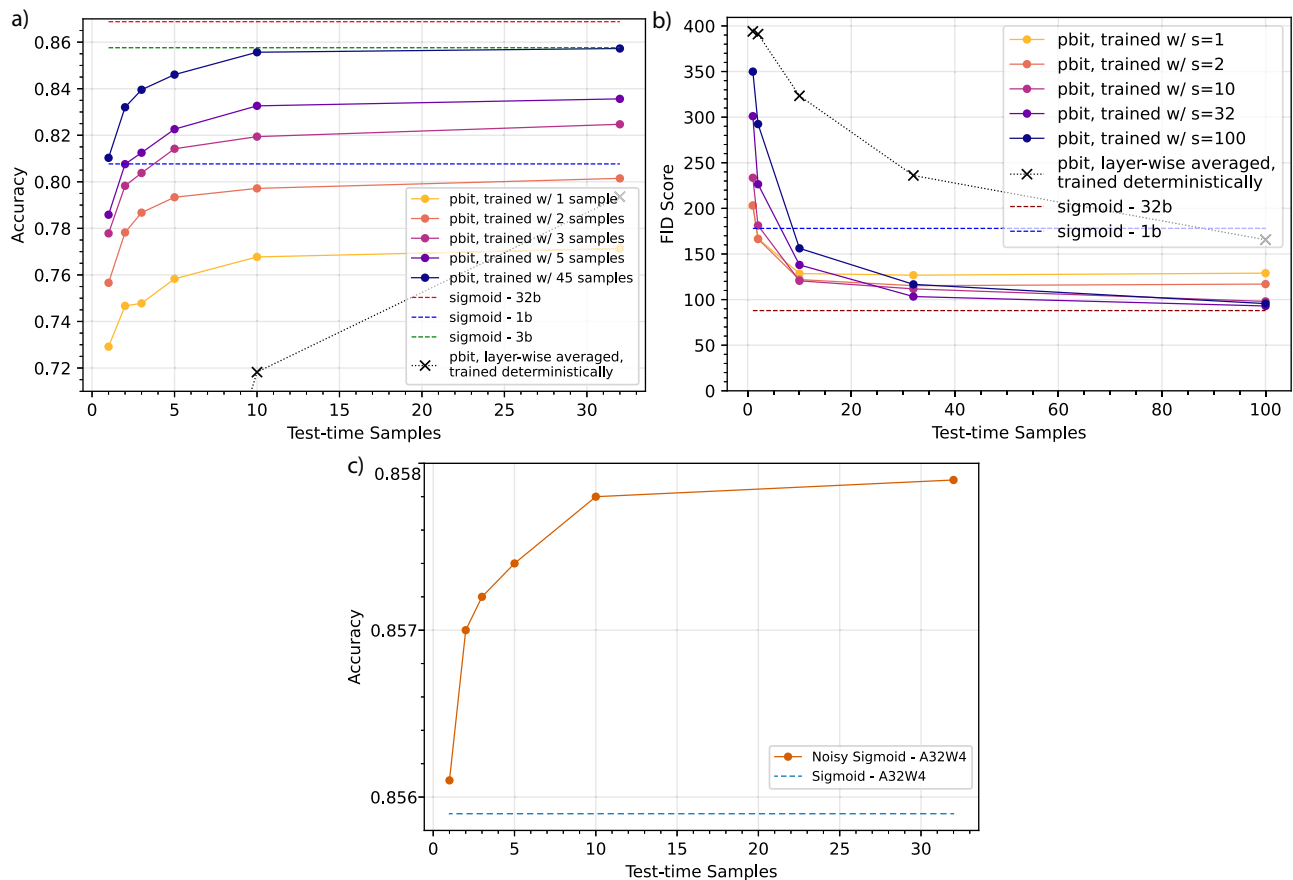


Fig. 4 | Quantitative performance evaluation of probabilistic and deterministic models on image classification and generation tasks. a Comparison of accuracy for CIFAR10 classification. **b** Comparison of FID metrics for a variational

autoencoder trained on the CelebA dataset. **c** Accuracy on CIFAR10 of a deterministic baseline compared with a model with noisy activations.

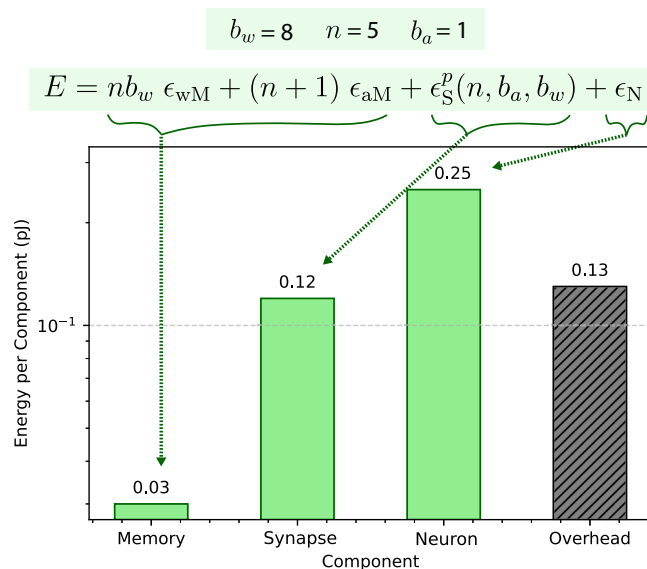


Fig. 5 | Energy cost of component operations of building block for QMC from 65 nm ASIC implementation reported in ref. 1.

very desirable, as it would have little impact on the end-to-end energy cost of inference.

- As shown in Fig. 6a, p-DNNs using p-bit activations show dramatic improvement in synapse energy owing to the reduction of activations to 1 bit, the MAC to AC benefit.

Table 1 | Variables for p-DNN analysis

Symbol	Description	Quantities for Fig. 6
N	Number of neurons	58,629
n	Fan-in to processing element	213
T	Number of samples	1
b_w	Bits per weight	32
b_a	Bits per activation	32, 1
ϵ_{wM}	Energy to access 1b from weight memory	$3.97 \frac{\text{pJ}}{\text{bit}}$
ϵ_{aM}	Energy to access 1b from activation memory	$0.0384 \frac{\text{pJ}}{\text{bit}}$
ϵ_{mul}	Energy to multiply b_w and b_a	8.989 pJ
ϵ_{add}	Energy to add $b_w + b_a$ numbers	0.239 pJ
ϵ_{act}	Energy for activation	0.087 pJ
ϵ_{rng}	Energy for RNG	0.181 pJ
$\epsilon_{compare}$	Energy for RNG-activation comparison	0.022 pJ

- The first sample from a probabilistic DNN will be more costly than subsequent samples (see Fig. 6b). The energy cost of loading the weights from memory is amortized across multiple samples, but is felt entirely for 1-sample inference. Intermediate activation tensors will be stored back in activation memory in a $T \times b_a$ memory chunk, and each layer will overwrite the same memory with new activation samples. Figure 6
- Figure 6c shows that tolerance for a 1% accuracy drop-off can come with a 2x+ reduction in energy used, going from a 32b DNN to p-DNN. We use accuracy per number of samples from the VGG5

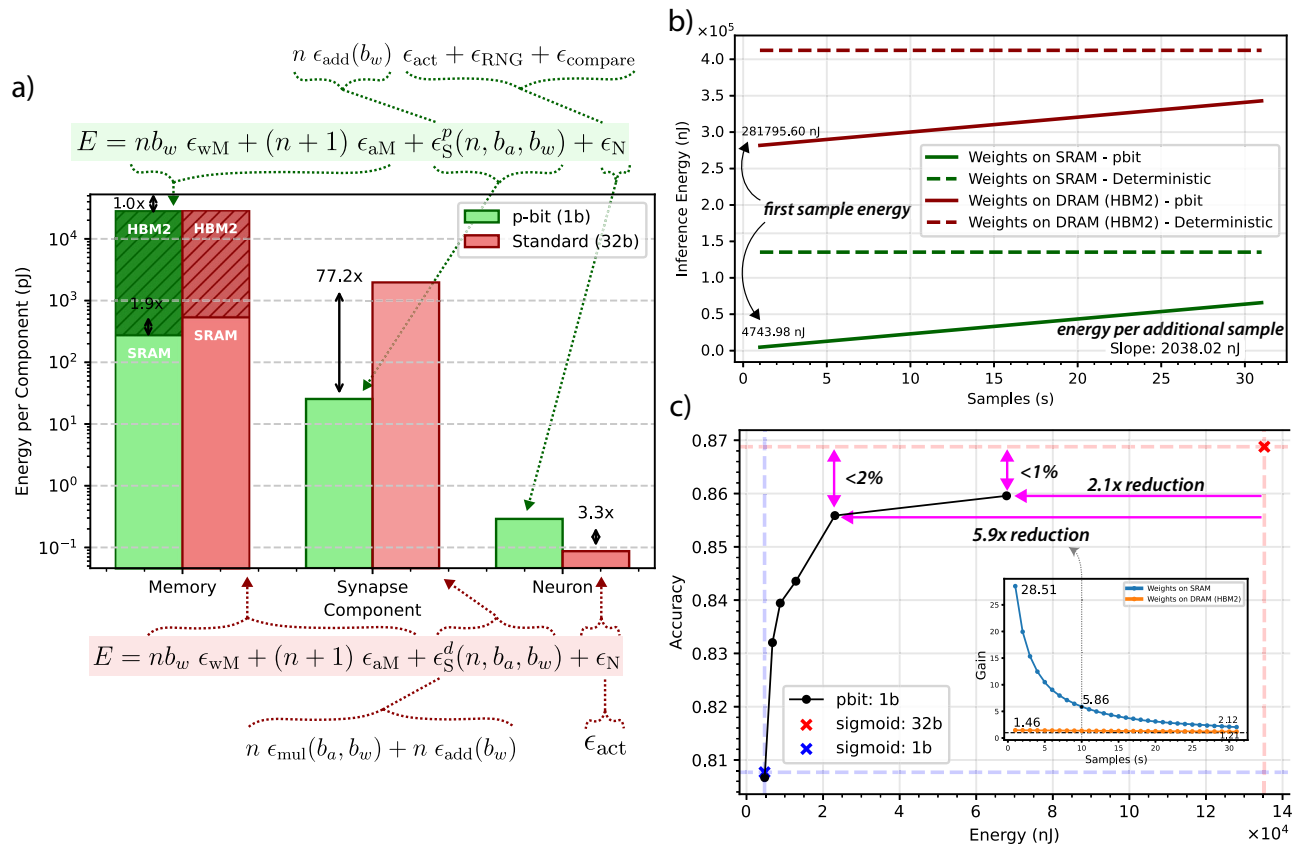


Fig. 6 | Energy cost analysis for probabilistic versus deterministic deep neural networks. **a** Energy of component operations of building block for DNN p-bit and deterministic building blocks. n is the average number of inputs to a nonlinearity across all layers of the image generation DNN, scaled to match the dimensions of ResNet50. The SRAM components use $\epsilon_{aM} = \epsilon_{wM} = 0.0384$ pJ/bit based on 1024-

word, 144-bits/word access to 65 nm TSMC process. The HBM2-DRAM components use $\epsilon_{aM} = 3.97$ pJ/bit²⁴, while retaining the activations on SRAM. **b** Shows energy per sample for the same setup as described for (a). **c** Combines energy per sample with accuracy per sample to show accuracy vs energy for the same setup as described for (a).

experiment in Fig. 4a and combine it with the energy per sample values in the inset of Fig. 6c to show an energy vs. accuracy curve comparing probabilistic and deterministic DNNs. From a purely computational perspective, though quantized models (like the 3b model matching performance of 10 p-bit samples) are competitive in energy iso-accuracy to stochastic models, it is important to remember that p-DNNs are *run-time adjustable*. More or fewer samples can be drawn at run-time to fit the application at hand, dynamically adjusting energy and accuracy to suit the user's needs.

Expanding on the framework

The real power of this framework is how easily it can be tweaked and expanded on as needed. For example, this framework can be tweaked to find the energy per inference for an entire model, even for models that may not traditionally be thought of as breaking down neatly into *synapse* + *neuron* elementary operations. For instance, Large Language Models (LLMs) like Llama have various nonlinearities in addition to more traditional neurons like the SiLU, including but not limited to RMSNorm, softmax, and ROPE. The last term (ϵ_N) of Eq. (1) can be split into multiple neuron types as needed to cover these various nonlinearities. The energy for the entire model can be defined by scaling weight reads by the number of weights across the model, scaling synapse energy by the number of macs across the model, and scaling each neuron type's energy by the number of those neurons in the overall model.

From Eq. (1) then, one would arrive at the following function for the energy per frame of inference (or equivalently for an LLM, Joules/Token):

$$E_{\text{frame}} = n_{\text{weights}} b_w \epsilon_{wM} + T(n_{\text{mac}} (b_a \epsilon_{aM} + E_{\text{synapse}}) + n_{\text{neurons}_1} (E_{\text{neuron}_1} + b_a \epsilon_{aM}) + \dots + n_{\text{neurons}_N} (E_{\text{neuron}_N} + b_a \epsilon_{aM})) \quad (4)$$

where there may be N different types of nonlinearities in the model. One may want to test a model where different activations are stored in different memories, i.e. ϵ_{aM} could be unique to each neuron type. This could be applied to the KV cache, for instance, which might be larger or further from compute than other activation buffers. The granularity with which an estimation can be made can be adjusted as desired.

Comparing FPGA implementations of DNN and p-DNN

We implement both a P-DNN with p-bit activations and a traditional DNN with sigmoid activations in an FPGA to validate the energy framework. FPGA-based hardware implementations of probabilistic models outside of stochastic computing approaches have dealt exclusively with energy-based models^{6–8}. On the other hand, this is not an energy-based application, though the general architecture is intentionally kept very similar to existing p-bit hardware solutions, as seen in Fig. 7. In keeping the design similar to existing p-bit hardware solutions, a few key decisions were taken:

Fig. 7 | FPGA Implementation architecture and functional validation of the probabilistic building block. a The processing element architecture for a building block, as implemented on the FPGA. **b** Image samples generated by the FPGA with 1 probabilistic sample.

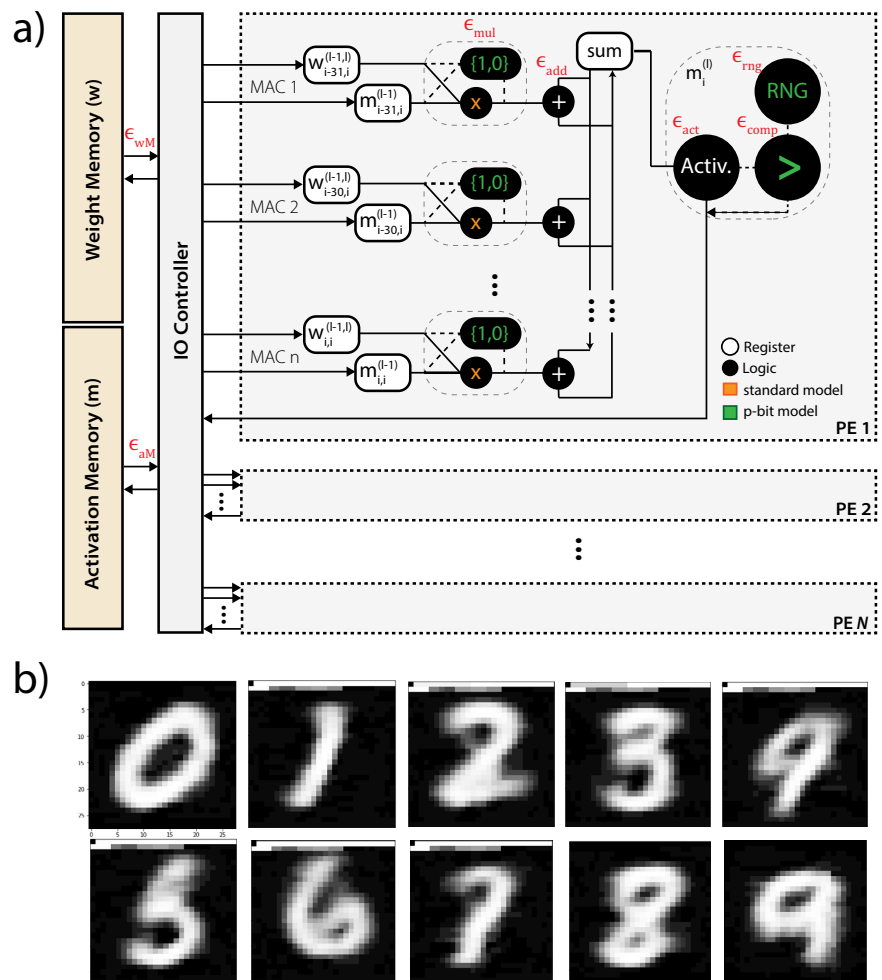


Table 2 | Energy and utilization for FPGA designs on the ZCU104

	Power (W)	Time (ns)	Energy (nJ)	LUTs	Regs
Full design					
p-bit	0.066	2.04e6	1.35e5	3796	2926
tanh	0.168	2.04e6	3.43e5	4158	2759
LFSR (RNG)					
p-bit	1.4e-4	–	–	16	33
Comparator					
p-bit	3.3e-4	–	–	17	33

- Only fully connected layers are implemented, as opposed to convolutional layers that may require sliding windows.
- The fan-in to any neuron in the network is limited to 32, similar to how, e.g., k-nearest neighbor designs for Boltzmann machines limit fan-in to k.

The image generation task is implemented via a DNN based on a Variational Autoencoder, shown in Fig. 9c, trained on the MNIST dataset. This DNN is mapped onto a hardware processing element as described by Fig. 7a. Two versions of this DNN are trained, one that uses p-bits and one that uses standard 32b σ activations, and consequently, two versions of the hardware are built. There are 2 differences between the designs. First, the *synapse* module implements a MAC operation for the standard design as opposed to AC for the p-bit design. Second, the *neuron* module skips the RNG and comparison operations for the standard design. These changes are summarized in Fig. 7.

The power and energy of these designs are compared in Table 2. We observe a 4.42x improvement in the energy consumption of the mac module alone, which leads to an overall energy improvement of 2.55x. This aligns with the gain prediction in Fig. 8, where a 1.875 pJ/bit memory energy (approximate energy/bit for BRAM on the ZU7EV fpga) corresponds to 2.3x gain, aligning closely with our experimentally obtained gain of 2.5x. With more efficient memory, this gain could be much larger as shown in Fig. 8b, c, motivating the incorporation of in-memory-compute architectures. We also observe that mac-related CLB LUT usage goes down by 2.9x.

A key consideration of using probabilistic models is the overhead that comes from the RNG over deterministic models. We see that the RNG and comparison overhead are negligible in terms of energy and area compared to the rest of the inference pipeline in custom hardware, which will only grow smaller as n grows larger.

While the absolute gains observed on FPGA are conservative by design, they validate the scaling trends predicted by the energy framework, which become increasingly favorable as models grow larger and memory energy dominates, as in contemporary LLM-scale workloads. We also note that the FPGA implementation is intended to validate the energy decomposition and scaling behavior predicted by the analytical framework, rather than to benchmark accuracy. Functional correctness was verified through generated outputs, while accuracy comparisons are reported using high-precision GPU simulations to avoid hardware-specific numerical confounds.

Discussion

The sample-aware training algorithm, paired with the energy analysis in Fig. 6, opens avenues to interesting probabilistic compute architectures that could optimize for latency or energy efficiency. One possibility is the

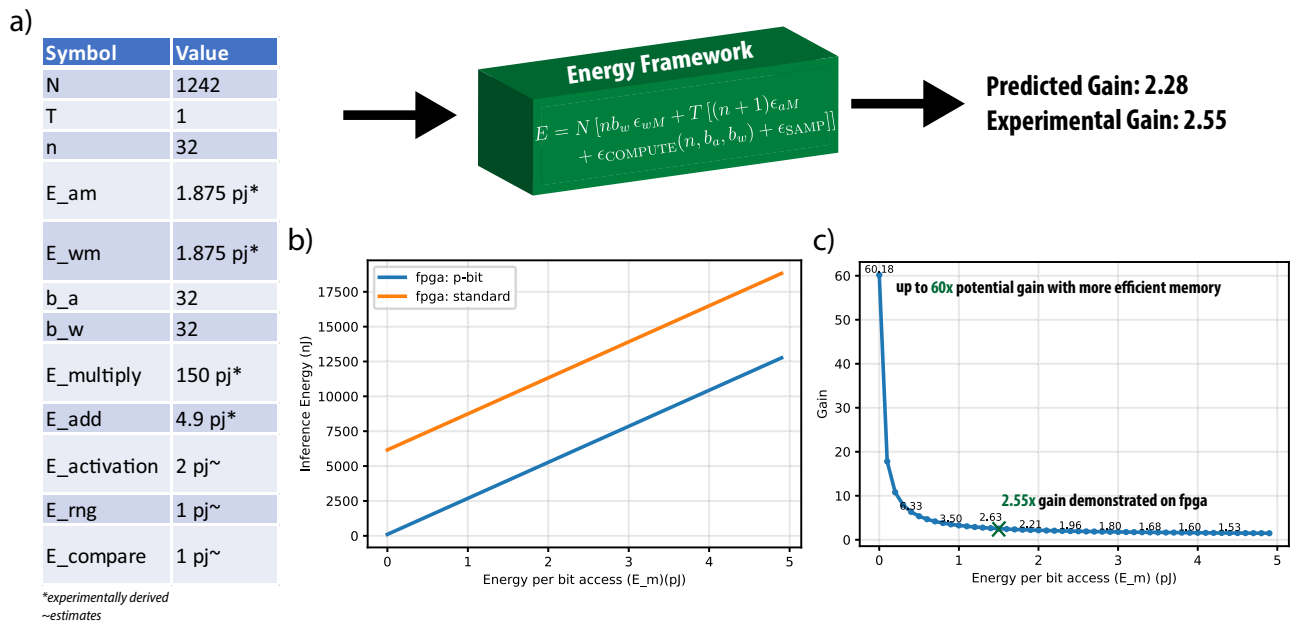


Fig. 8 | Energy analysis for FPGA Implementation. **a** Contains values used for the FPGA design's energy prediction. **b** Is the predicted energy per frame of inference on the FPGA. **c** Shows the predicted gain curve comparing probabilistic and deterministic models across changing memory efficiency ($E_{wM} = E_{aM} = E_M$).

drawing of samples in parallel across separate hardware units, so long as each sample is independent and identically distributed (i.i.d), which they are for these models, and combining the samples at the output of the last layer. The internal math is all 1-bit, but the result could be the equivalent of 3-bit accuracy, delivered in less time. Such a hardware system could be a perfect fit for latency-sensitive situations where energy and resources are not a concern. This is an opportunity that quantization cannot take advantage of.

Another possibility for taking advantage of sampling is to build an n -bit p-DNN, which would match the energy of an n -bit DNN to within the difference in neuron energy ($\epsilon_{\text{rng}} + \epsilon_{\text{compare}}$). Adding an additional sample to the inference pipeline could lead to 2+ % in accuracy, while increasing energy by just 0.7% (see Fig. 6b) for models that run on DRAM. We consider our estimate conservative in that larger, production AI models, like LLMs, would use significantly more memory-related energy, making the additional sampling energy minuscule.

An important advantage to doing inference via probabilistic systems over direct quantization is the ability to adjust the energy and accuracy at run-time through the number of samples. We show that a 1-bit probabilistic inference engine can be the equivalent of a 1-bit, 2-bit, and 3-bit quantized inference engine rolled into one. While each comparison on its own leads to roughly parity energy use, this is a giant practical leap, as a single hardware can serve all 3 bit-widths.

These unique aspects of probabilistic inference extend from the foundation of probabilistic computing. The FPGA experiment embodies the connection of this work to its motivating foundations. It shows that the same building block used for Boltzmann machines can implement p-DNNs. The idea is to show that p-DNNs can be implemented on near-BM hardware, for which there has already been extensive work done to build devices for and optimize at the system level^{7,9,10}.

How is it that machinery designed for Boltzmann machines can be applied to feed-forward DNNs? Graph colored Ising models, for instance, enforce updating a subset of the total neurons per timestep such that every neuron in a model with N_c colors would have an attempt to flip in N_c timesteps. The neurons of each color could be thought of as a layer in a feed-forward neural network; an MLP with p-bit activations could be seen as a rearranged graph-colored Ising model. In the same way, Ising models with limited connectivity per neuron are able to be mapped to efficient in-memory-compute architectures¹¹, limited connectivity MLPs could be mapped to in-memory-computing architectures for ML tasks. The right

balance between connectivity and performance could be application-specific. An important benefit to implementing p-bits in a feed-forward fashion is that the backpropagation algorithm can be used to train the model, as opposed to standard energy-based training schemes like Hebbian learning or contrastive divergence, which are notably less efficient.

While preliminary works into probabilistic machine learning exist, they are not motivated by mapping their designs to unconventional computing substrates, and importantly, do not take advantage of sampling during inference. While¹² state that stochastic inference surpasses HP models within 25 samples for CIFAR10 classification, we find that accuracy comes to within 1% but does not quite match. On the other hand, models implementing deterministic inference have found great success. For vision applications, W1A1 models have proven to be quite commendable^{2,13,14}. Most recently,¹⁵ have set state-of-the-art results using W1A1 models for vision applications. Stochastic inference for feed-forward CNNs was studied in¹² but we were not able to replicate their results.

While the experiments shown in this work demonstrate probabilistic inference on vision models, the ideas, and the presented energy framework, easily extend to Large Language Models (LLMs). Feed-forward machine learning models are at the core of what makes AI so powerful today. While high bit weight and activation models (W32A32) were common a decade ago, the energy cost of running today's AI models has skyrocketed. The energy benefits of going to $W \times A_y$ models where x and y are small, are clear, making it a highly active area of research. Recent works have reduced weight precision in large language models (LLMs) to 4 bits and beyond^{16,17}, but activation precision has remained challenging to reduce below 8 bits due to accuracy falloff. More so than weights, activations need to be adaptable at runtime, able to handle large-scale outliers that may arise during inference^{18,19}. This is where p-bits may provide a means to scale down LLM activation bits without compromising on accuracy. A 4-bit activation LLM for instance may achieve the required accuracy using a few samples. With fine-tuning and retraining methods like sample-aware training, that accuracy may be boosted, motivating research in that direction. While this work focuses on inference energy, extending sample-aware training to reduce training-time energy—particularly for large language models—is an important direction for future work and is not claimed as a contribution here.

We'd like to note that, as it stands, the energy efficiency of p-DNNs, similar to standard DNNs, will depend more on memory technology

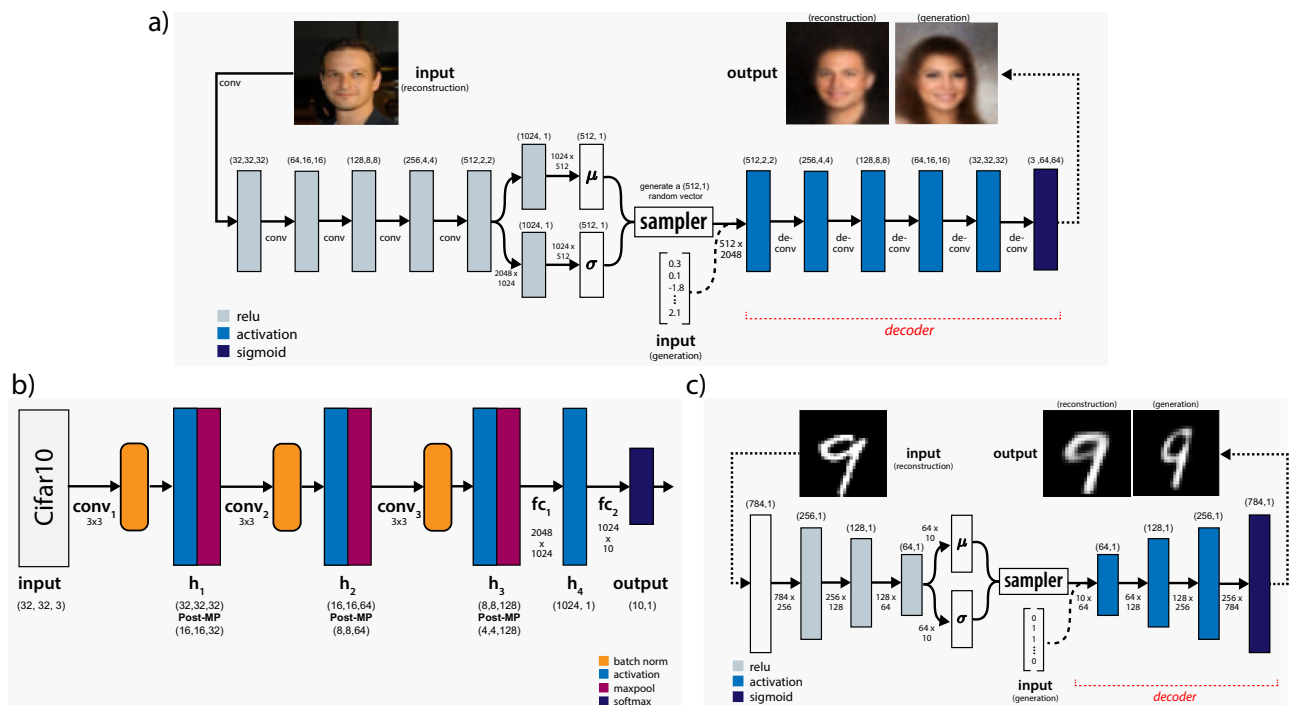


Fig. 9 | Architectures of models used for image generation and classification experiments. a Model for celebrity image generation experiment. **b** Model for CIFAR10 classification experiment. **c** Model for MNIST generation FPGA experiment.

compared to changes in other facets (tech-node, model architecture, etc.). In order for p-bits to outperform standard quantization on a purely *compute energy* front (synapse + neuron energy), advancements in sampling-based training schemes and implementations that enable drawing fewer samples will be critical. One example of this is finding more effective ways to combine different samples, beyond simple averages. The flip side to this, as we noted previously, is that additional compute on the same data, in the vein of sampling, can improve accuracy without any notable impact on end-to-end energy.

P-bits are a versatile tool with a proven track record in building highly energy-efficient platforms for sampling and optimization problems, now applied to DNNs. We discuss two compute-only sampling schemes that can improve on the accuracy of standard DNNs with minor end-to-end energy penalties, demonstrated on image classification and generation. We pair this with an energy framework for probabilistic systems based on the basic building block consisting of a p-bit and its synapse. This building block is mapped to the p-bit-based ASIC in ref. 1 to reveal its energy components. We further support our framework with an FPGA demonstration and show closely matching energy improvements to the framework's prediction. Conceptually, this work establishes that probabilistic computing primitives—previously confined to energy-based and optimization-centric models—can be systematically extended to feedforward deep learning, enabling a unified hardware substrate for both sampling-based inference and gradient-trained neural networks. Future work could study analog implementations for p-bit-based DNNs, integrating with in-memory compute architectures, or extending these results beyond vision applications.

Methods

The following are simulation details for training the DNNs used in this manuscript. Namely, this includes the classification model and two image generation models, one of which was run on the FPGA. Code is available on request.

For classification experiments, we went with sigmoid for the p-bit implementation to give deterministic models the benefit of the doubt, as

sigmoid activations performed much better for deterministic models. For image generation, we found no notable difference in performance between sigmoid and tanh, and so stuck with tanh p-bits and tanh deterministic baselines. For each experiment, the derivative of the p-bit was implemented as the derivative of the activation, going “straight-through” the sign function²⁰. All models were trained on an RTX 4070 Super, and Wandb was used to log and track experiments.

Classification experiment

The classification experiment was carried out on the CIFAR-10 dataset. This dataset contains (32,32,3) dimensional images that span 10 classes: plane, car, bird, cat, deer, dog, frog, horse, ship, truck. The data is split between 50000 training images and 10000 test images.

The architecture is depicted in Fig. 9. All models are trained for 1000 epochs using the ADAM optimizer on cross-entropy loss using pytorch. All models use a batch size of 64 and have a constant learning rate of 1e-3. There is a standard horizontal flip pre-processing on the dataset during training for all models. Batch norm can be folded into the preceding convolutional or fully connected layer during inference²¹, allowing us to take advantage of it during training without any penalty on inference energy.

The CIFAR10-classification models are initialized with weights following Glorot (Xavier) weight initialization from ref. 22. Glorot initialization is a one-time overhead that ensures that the weights of the model have the same variance per layer despite changing layer configurations, and are centered around 0. It can be described as follows.

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{\text{fan_in} + \text{fan_out}}\right) \quad (5)$$

where i and j are the input and output units of weight matrix W . fan_in describes the input size of weight layer W and fan_out describes the output size. Biases are set to 0.

Celebrity generation experiment

The images shown in Fig. 3 are from a variational autoencoder that was trained on the CelebA dataset²³. This dataset consists of 202,599 face images of dimension $178 \times 218 \times 3$, of which 162,770 images were used as the training set, and 19,962 images were used as the test set (the remaining 19,867 validation images were not used). The images were scaled down to $64 \times 64 \times 3$ via `torch.resize()` to speed up training and make experimentation more feasible.

The VAE uses 5 convolutional layers for the encoder (3, 32, 64, 128, 256, 512) with kernel size 4, stride 2, padding 1, and the same layers in reverse as transposed convolutions for the decoder. A latent dimension of 512 was used. Training used binary cross-entropy loss paired with the ADAM optimizer. Training was carried out for 1000 epochs, using a learning rate of $1e-4$ and a batch size of 64.

FPGA generation experiment

A Variational Autoencoder (VAE) is trained to generate the MNIST digits following the architecture in Fig. 9c. The deterministic and probabilistic versions of this design are implemented on a ZCU104 board, which houses a Zynq Ultrascale+ ZU7EV FPGA. The weights for all models are converted to fixed-point Q6.25. The p-bit design uses a single sample, which proved sufficient for generating the images. The design was run at 100 MHz. The input is deterministically binarized before feeding into the first layer of the model. The full VAE was trained using a GPU, and the decoder was implemented on the FPGA. For the MAC operation, we implement a high precision intermediate accumulation as is standard in FMAC units.

Data availability

Code for these experiments is available upon request from the corresponding author. All data used were part of publicly available ML datasets.

Received: 7 October 2025; Accepted: 26 February 2026;

Published online: 27 April 2026

References

- Li, M.-C. et al. 12.2 p-Circuits: Neither Digital Nor Analog. In *2025 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 68, 1–3. <https://ieeexplore.ieee.org/abstract/document/10904553> (2025).
- Hubara, I., Courbariaux, M., Soudry, D., El-Yaniv, R. & Bengio, Y. Quantized neural networks: training neural networks with low precision weights and activations. <https://doi.org/10.48550/arXiv.1609.07061> (2016).
- Raiko, T., Berglund, M., Alain, G. & Dinh, L. Techniques for learning binary stochastic feedforward neural networks. <http://arxiv.org/abs/1406.2989>. (2015).
- Capra, M. et al. Hardware and software optimizations for accelerating deep neural networks: survey of current trends, challenges, and the road ahead. *IEEE Access* **8**, 225134–225180 (2020).
- Jacob, B. et al. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2704–2713. <https://ieeexplore.ieee.org/document/8578384> (2018).
- Pervaiz, A. Z., Sutton, B. M., Ghantasala, L. A. & Camsari, K. Y. Weighted p-bits for FPGA implementation of probabilistic circuits. *IEEE Trans. Neural Netw. Learn. Syst.* **30**, 1920–1926 (2019).
- Sutton, B. et al. Autonomous probabilistic coprocessing with petaflips per second. *IEEE Access* **8**, 157238–157252 (2020).
- Singh, N. S. et al. Hardware Demonstration of Feedforward Stochastic Neural Networks with Fast MTJ-based p-bits. In *2023 International Electron Devices Meeting (IEDM)*, 1–4 <https://ieeexplore.ieee.org/document/10413686> (2023).
- Aadit, N. A. et al. Massively parallel probabilistic computing with sparse ising machines. *Nat. Electron.* <http://arxiv.org/abs/2110.02481> (2022).
- Daniel, J. et al. Experimental demonstration of an on-chip p-bit core based on stochastic magnetic tunnel junctions and 2D MoS₂ transistors. *Nat. Commun.* **15**, 4098 (2024).
- Chowdhury, S., Camsari, K. Y. & Datta, S. Accelerated quantum Monte Carlo with probabilistic computers. *Commun. Phys.* **6**, 1–9 (2023).
- Li, Y. et al. Binary-stochasticity-enabled highly efficient neuromorphic deep learning achieves better-than-software accuracy. *Adv. Intell. Syst.* **6**, 2300399 (2024).
- Courbariaux, M., Bengio, Y. & David, J.-P. BinaryConnect: training deep neural networks with binary weights during propagations <http://arxiv.org/abs/1511.00363> (2016).
- Courbariaux, M., Hubara, I., Soudry, D., El-Yaniv, R. & Bengio, Y. Binarized neural networks: training deep neural networks with weights and activations constrained to +1 or -1. <http://arxiv.org/abs/1602.02830> (2016).
- Shi, X., Sun, C., Qi, Z., Hao, L. & Yang, X. Expanding-and-shrinking binary neural networks. <http://arxiv.org/abs/2503.23709> (2025).
- Unsloth. unsloth/DeepSeek-V3-0324-GGUF hugging face. <https://huggingface.co/unsloth/DeepSeek-V3-0324-GGUF> (2025).
- Ma, S. et al. The Era of 1-bit LLMs: all large language models are in 1.58 Bits. <http://arxiv.org/abs/2402.17764> (2024).
- Xiao, G. et al. SmoothQuant: accurate and efficient post-training quantization for large language models. <http://arxiv.org/abs/2211.10438> (2024).
- Dettmers, T., Lewis, M., Belkada, Y. & Zettlemoyer, L. LLM.int8(): 8-bit matrix multiplication for transformers at scale. <http://arxiv.org/abs/2208.07339> (2022).
- Shekhtovsov, A. & Yanush, V. Reintroducing straight-through estimators as principled methods for stochastic binary networks. <http://arxiv.org/abs/2006.06880> (2021).
- Yvinec, E., Dapogny, A. & Bailly, K. To fold or not to fold: a necessary and sufficient condition on batch-normalization layers folding. <http://arxiv.org/abs/2203.14646> (2022).
- Glorot, X. & Bengio, Y. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*. (PMLR, 2010).
- Liu, Z., Luo, P., Wang, X. & Tang, X. Deep learning face attributes in the wild. <http://arxiv.org/abs/1411.7766> (2015).
- O'Connor, M. et al. Fine-grained DRAM: energy-efficient DRAM for extreme bandwidth systems. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*, 41–54 <https://dl.acm.org/doi/10.1145/3123939.3124545> (ACM, Cambridge, MA, 2017).

Acknowledgements

This work was partially supported by the ONR-MURI Grant No. N000142312708, OptNet: Optimization with p-Bit Networks.

Author contributions

L.G. set up the experiments and machine learning pipelines, conducted and recorded training and inference experiments, and built the FPGA design. M.L. and S.S. provided an energy breakdown of a QMC-ASIC and conducted additional simulations to get energy estimates for core operations on a TSMC-65nm platform. All authors were critical in establishing the probabilistic methods discussed and demonstrated in the manuscript. L.G. and S.D. lead the writing of the paper, with contributions/revisions from all authors. All authors read and approved the final manuscript.

Competing interests

L.G., S.D., and B.B. have financial interests in Ludwig Computing. S.S. has a financial interest in Ixana.

Additional information

Supplementary information The online version contains supplementary material available at

<https://doi.org/10.1038/s44335-026-00063-7>.

Correspondence and requests for materials should be addressed to Lakshmi A. Ghantasala.

Reprints and permissions information is available at <http://www.nature.com/reprints>

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2026